PROYECTO ENTREGABLE · HTML Y CSS

El sitio web de un negocio local

Construye, documenta y publica tu primer sitio para un cliente

Nombre: _____ Fecha: _____

1. El encargo

La situación

Muchos negocios de barrio venden solo por WhatsApp o Instagram: no tienen dónde mostrar sus productos, sus precios ni cómo llegar, y pierden clientes porque no alcanzan a responder a todos.

Elige un negocio real de tu barrio o invéntalo (una panadería, un taller de motos, un hostal, una veterinaria, una tienda de ropa, una barbería…) y construye su sitio web, como si te hubieran contratado.

El cliente quiere un sitio sencillo que se vea bien en el celular, que transmita confianza y que permita contactarlo o hacer un pedido.

No hay un diseño único correcto: tú decides los colores, las fuentes y la organización. Lo que se evalúa es que uses bien lo visto en el curso y que **demuestres por qué tu código funciona**. Por eso el proyecto se entrega en un repositorio público con un documento técnico (sección 4).

2. Qué debe tener el sitio

Dos páginas como mínimo, con el mismo encabezado, navegación y pie de página:

- **index.html**: presentación del negocio, sus productos o servicios (al menos 3, cada uno como tarjeta), una sección con información útil en lista (horarios, pasos, recomendaciones…), opiniones de clientes y cómo llegar (mapa o video).
- **Una segunda página** (precios, reservas, pedidos o contacto): una tabla de precios o tarifas y un formulario para contactar o pedir.
- Navegación entre las dos páginas y anclas a las secciones.
- Que se pueda usar cómodamente en un celular de 360 px de ancho.

3. Elementos obligatorios

Puedes usar más cosas de las vistas en clase, pero estas deben aparecer y estar **bien usadas**, no puestas solo para cumplir.

HTML

Grupo	Debe aparecer
Documento	`<!DOCTYPE html>`, `lang`, `meta charset`, `meta viewport`, `title` y `description` en cada página.
Semántica	`header`, `nav`, `main`, `section`, `article`, `footer` (y `aside` si tiene sentido). Un solo `h1` por página.

Grupo	Debe aparecer
Texto	`strong`, `em`, `s` (por ejemplo, un precio anterior) y `abbr` o `sup`.
Enlaces y medios	Rutas relativas, un ancla entre páginas, un enlace externo en otra pestaña, `img` con `alt` y un `iframe` o `video`.
Datos	Una `table` con al menos un `colspan` o `rowspan`. Una lista `ul` y una `ol`.
Citas	Las opiniones con `blockquote` y `cite`.
Formulario	`form` con `action` y `method`, `label` por campo, al menos tres `type` distintos, un `textarea` y botón de envío.

CSS

Grupo	Debe aparecer
Organización	Una hoja externa compartida por las dos páginas, variables en `:root` usadas en toda la hoja, comentarios por sección. Sin `!important`.
Color y fondos	Al menos un color con canal alfa y un gradiente. Un fondo con imagen o fondo múltiple.
Tipografía y texto	`font-family` con respaldo, tamaños en `rem`/`em`, `line-height`, `letter-spacing` o `text-transform`, `text-decoration` y control de texto largo (`overflow-wrap`).
Modelo de cajas	`padding`, `margin` (incluido un centrado con `margin: auto`), `border` por lado, `border-radius`, `max-width` y `overflow`.
Tablas y listas	`border-collapse`, `list-style` y `::marker`.
Selectores	Clases con nombres BEM, un selector de atributo, un combinador, una pseudoclase estructural (`:nth-child`, `:first-child`…) y `::before` o `::after`.
Interacción	`:hover`, `:focus-visible` y `:target` o `:focus-within`.
Reglas	Un `@media` para celular y una animación con `@keyframes`.

4. Documentación técnica (README.md)

En el trabajo real, el código va acompañado de documentación para que otra persona lo entienda y lo pueda mantener. Crea un archivo `README.md` en la raíz del repositorio con estas secciones. Usa **ejemplos de tu propio código**, no definiciones copiadas. El README es tu sustentación: todo lo que quieras demostrar debe estar ahí.

Sección	Qué debes explicar
1. El proyecto	Qué negocio es, para quién es el sitio y qué necesita el cliente.
2. Estructura	Las carpetas, las páginas y por qué elegiste cada etiqueta semántica en las partes principales.
3. Sistema visual	Tus variables (colores, fuentes, espacios) y por qué las elegiste.
4. Especificidad	Al menos **3 conflictos reales** de tu hoja: las reglas que compiten, la especificidad de cada una calculada como (IDs, clases, etiquetas) y cuál gana. Uno de ellos debe resolverse por **orden** (empate). Cada conflicto con una **captura del inspector** del navegador (panel Estilos) donde se vea la regla que pierde tachada.
5. Herencia	Una propiedad que tus elementos heredan y una que no se hereda, con el lugar de tu sitio donde se nota.
6. Modelo de cajas	El cálculo del ancho total de uno de tus componentes (contenido + padding + borde) y un caso de márgenes que se solapan, si lo tienes.
7. Selectores	Cómo funciona tu pseudoclase estructural: qué hermanos cuenta y a cuál elemento termina aplicando.
8. Responsive	Qué cambia con tu `@media` y por qué.
9. Problemas y soluciones	Al menos dos cosas que no funcionaron a la primera, por qué pasaba y cómo lo arreglaste.
10. Capturas	Las dos páginas en escritorio y a 360 px de ancho, insertadas en el README con rutas relativas a la carpeta `capturas/`. Si publicas el sitio con GitHub Pages, agrega el enlace.

Ejemplo de cómo documentar un conflicto

Este ejemplo usa reglas inventadas; en tu README van las de tu sitio.

```
### Conflicto 1: el precio de la tarjeta en promoción

Reglas que compiten sobre el mismo <p>:
  .tarjeta p                → (0,1,1)   color: var(--texto)
  .tarjeta--promo .precio   → (0,2,0)   color: var(--acento)

Gana .tarjeta--promo .precio porque tiene 2 clases contra 1 clase
y 1 etiqueta. No importa que esté antes o después en el archivo.

Al principio escribí solo .precio (0,1,0) y perdía contra .tarjeta p.
```

> 💭 **Para pensar**
>
> **1.** Si dos reglas tienen exactamente la misma especificidad, ¿cuál se aplica?
>
> **2.** ¿Por qué un color puesto en un elemento padre no siempre llega a sus hijos?
>
> **3.** Si usas `:nth-child`, ¿cuentan los elementos ocultos o los que tienen otra etiqueta?
>
> **4.** ¿Por qué no basta con mirar el `width` para saber cuánto ocupa una caja?

5. Reglas del proyecto

- Todo el HTML y el CSS lo escribes tú, y tu README debe demostrar que lo entiendes.
- Sin `!important`, sin `<style>` y sin atributos `style`. Los conflictos se resuelven con especificidad, orden y herencia, y se documentan.
- No se exige Flexbox ni Grid: el diseño puede ser de una sola columna.
- Usa fotos y videos de bancos gratuitos (Pexels, Pixabay) o propios, guardados en tus carpetas.
- Si un dato es de un negocio real (dirección, teléfono), pide permiso al dueño o invéntalo.

Estructura sugerida

```
nombre-del-negocio/
├── index.html
├── segunda-pagina.html
├── README.md
├── css/styles.css
├── images/
├── video/
└── capturas/      ← imágenes que usa el README
```

Subirlo a GitHub y en plataforma

- Subirlo en un repositorio publico y subir el enlace en plataforma con su hash
- Hacerle deepwiki al finalizar
- Sino se hace deepwiki se califica sobre 80, si adicional no se sube a plataforma se califica sobre 70

6. Etapas y entregables

Etapa	Qué se entrega
1 · Propuesta	El negocio elegido, a quién va dirigido y un boceto en papel o digital de las dos páginas.
2 · Estructura	Las dos páginas en HTML con todo su contenido, sin estilos.
3 · Entrega final	El **enlace a tu repositorio público** de GitHub, con el proyecto completo y el `README.md` con sus 10 secciones y capturas.

7. Rúbrica

Criterio	Qué se revisa	Pts
HTML semántico y contenido	Documento completo, etiquetas semánticas bien elegidas, texto con significado, enlaces, rutas y multimedia	15
Tabla, listas, citas y formulario	Uso correcto de cada elemento obligatorio y de sus atributos	10
Organización y cascada en CSS	Hoja externa, variables, comentarios, sin !important	10
Diseño visual	Tipografía, color, fondos y modelo de cajas coherentes y legibles	15
Selectores e interacción	BEM, atributos, combinadores, pseudoclases y pseudoelementos bien usados	10
Responsive y animación	Funciona a 360 px sin desbordes; @media y @keyframes	10
Documentación (README.md)	Las 10 secciones con ejemplos de su propio código; cálculos de especificidad y del modelo de cajas correctos; capturas del inspector que respaldan cada conflicto	25
Repositorio	Público, ordenado como la estructura sugerida, imágenes del README visibles en GitHub y varios commits que muestran el avance	5
Total		100

8. Antes de entregar

- ☐ Las dos páginas abren con Live Server y todos los enlaces y anclas funcionan.
- ☐ Revisé cada fila de la tabla de elementos obligatorios.
- ☐ No hay `!important`, `<style>` ni atributos `style` (búscalo con Ctrl + F).
- ☐ Todas las imágenes tienen `alt` y el `iframe` tiene `title`.
- ☐ Probé el sitio a 360 px: nada se sale de la pantalla.
- ☐ Probé `:hover`, la tecla TAB y las anclas.
- ☐ Mi README tiene las 10 secciones y los conflictos usan reglas reales de mi `styles.css`.
- ☐ Cada conflicto tiene su captura del inspector y lo que muestra coincide con mi cálculo.
- ☐ Abrí el enlace de mi repositorio en una ventana de incógnito: es público y las imágenes del README se ven.